

/Users/mery/lectures/libraryframac/project-divers/exercices.tex

Table des matières

1	TD5	2
2	TD6	3
3	TD7	4
3.1	Annotations	4
3.2	Contrat division euclidienne	5
3.3	Contrat factorielle	6
4	TD8	7
4.1	Power2	7
4.2	Mutables, pointeurs etc	9

Cours MOdélisation, Vérification et Expérimentations
Exercices
Utilisation d'un environnement de vérification Frama-c (I)
par Dominique Méry
30 avril 2026

1 TD5

Exercice 1 *annotations03.c, annotations04.c*

Nous vous donnons des annotations que vous devez analyser avec Frama-c.

Listing 1 – annotations03.c

```
Question 1.1 #include <limits.h>  
/*@ requires a >= 0 && b >= 0;  
  @ assigns \nothing;  
  @ ensures \result == \old(a)+\old(b)-2;  
  
  @*/  
int annotations03(int a, int b)  
{  
  int x, y, z;  
  x = a;  
  /*@ assert l1: x == a; */  
  y = b;  
  /*@ assert l2: x == a && y == b; */  
  z = a+b-2;  
  /*@ assert l3: x == a && y == b && z==a+b-2; */  
  return(z);  
}
```

Listing 2 – annotations04.c

```
Question 1.2 /*@ requires a >= 0;  
  @ assigns \nothing;  
  @ ensures \result == 0;  
  @*/  
int annotation(int a)  
{  
  int x;  
  x = a;  
  return(x);  
}
```

Exercice 2 *annotations05.c*

Soit le petit programme suivant

Listing 3 – annotations05.c

```
void ex(void) {  
  
  int x=2,y=4,z,a;
```

```

/*@ assert x <= y;

x = x*x;
/*@ assert x == a*y && x* 2*x >= 8;
y = 2*x;
z = x + y;
/*@ assert z == x+y && x* y >= 8;
}

```

Analyser la correction des annotations avec Frama-c et trouver a pour que cela soit correctement analysé.

Exercice 3 Soit le petit programme suivant

Listing 4 – annotations06.c

```

void ex(void) {
  int x0,y0,z0;
  int x=x0,y=x0,z=x0*x0;
  /*@ assert l1: x == y && z == x*y;
  x = x*x;
  /*@ assert l2: x == y*y && z == x;
  y = x;
  /*@ assert l3: x + y + 2*z == (x0+x0)*(x0+x0) ;
  z = x + y + 2*z;
  /*@ assert z == (x0+x0)*(x0+x0);
}

```

Analyser la correction des annotations avec Frama-c et définir un contrat en supposant que l'on ajoute une instruction `return(z)`.

Exercice 4 annotation07.c

Soit le petit programme suivant

Listing 5 – annotations07.c

```

/*@
  assigns \nothing;
  ensures \result >= x && \result >= y && (\result == x || \result == y);
*/

int max ( int x, int y ) {

  if ( x >= y )
  {
    return x ;
  }
  return y ;
}

```

Analyser la correction des annotations avec Frama-c. Ce programme calcule la valeur du maximum de deux valeurs. Il faudra définir correctement le contrat.

2 TD6

Exercice 5 La définition structurelle des transformateurs de prédicats est rappelée dans le tableau ci-dessous :

S	$wp(S)(P)$
$X := E(X, D)$	$P[e(x, d)/x]$
SKIP	P
$S_1; S_2$	$wp(S_1)(wp(S_2)(P))$
IF B S_1 ELSE S_2 FI	$(B \Rightarrow wp(S_1)(P)) \wedge (\neg B \Rightarrow wp(S_2)(P))$

- Axiome d'affectation : $\{P(e/x)\} X := E(X) \{P\}$.
- Axiome du saut : $\{P\} \text{skip} \{P\}$.
- Règle de composition : Si $\{P\} S_1 \{R\}$ et $\{R\} S_2 \{Q\}$, alors $\{P\} S_1; S_2 \{Q\}$.
- Si $\{P \wedge B\} S_1 \{Q\}$ et $\{P \wedge \neg B\} S_2 \{Q\}$, alors $\{P\} \text{if } B \text{ then } S_1 \text{ then } S_2 \text{ fi} \{Q\}$.
- Si $\{P \wedge B\} S \{P\}$, alors $\{P\} \text{while } B \text{ do } S \text{ od} \{P \wedge \neg B\}$.
- Règle de renforcement/affaiblissement : Si $P' \Rightarrow P$, $\{P\} S \{Q\}$, $Q \Rightarrow Q'$, alors $\{P'\} S \{Q'\}$.

Question 5.1 Simplifier les expressions suivantes :

1. $WP(X := X+Y+7)(x+y=6)$
2. $WP(X := X+Y)(x < y)$

Question 5.2 On rappelle que $\{P\} S \{Q\}$ est défini par l'implication $O \Rightarrow WP(S)(Q)$. Pour chaque point énuméré ci-dessous, monter que la propriété $\{P\} S \{Q\}$ est valide ou pas en utilisant la définition suivante :

$$\{P\} S \{Q\} = P \Rightarrow WP(S)(Q)$$

1. $\{x+y = 7\} X := Y+X \{2 \cdot x+y = 6\}$
2. $\{x < y\} \text{IF } x \neq y \text{ THEN } x := 5 \text{ ELSE } x := 8 \text{ FI} \{x \in \{5, 8\}\}$

Question 5.3 Utiliser frama-c pour vérifier les éléments suivants :

1. $\{x+y = 7\} X := Y+X \{2 \cdot x+y = 6\}$
2. $\{x < y\} \text{IF } x \neq y \text{ THEN } x := 5 \text{ ELSE } x := 8 \text{ FI} \{x \in \{5, 8\}\}$

3 TD7

3.1 Annotations

Exercice 6 Annotations

On rappelle que l'annotation suivante du listing 6 est correcte, si les conditions suivantes sont vérifiées :

- $pre(v_0) \wedge v = v_0 \Rightarrow A(v_0, v)$
- $pre(v_0) \wedge B(v_0, v) \Rightarrow post(v_0, v)$
- $A(v_0, v) \Rightarrow wp(v = f(v))(B(v_0, v))$ où $wp(v = f(v))(B(v_0, v))$ est définie par $B(v_0, v)[f(v)/v]$.

Dans le cas de Frama-c, la valeur initiale d'une variable v est notée $\backslash at(v, Pre)$ et aussi $\backslash old(v)$. Nous utiliserons la notation v_0 dans cet exercice.

Listing 6 – contrat

```
requires pre(v)
ensures post(\old(v), v)
type1 truc(type2 v)
/*@ assert A(v0, v); */
v = f(v);
/*@ assert B(v0, v); */
return val;
```

Soient les annotations suivantes. Les variables sont supposées de type int.

Question 6.1 *annotations11.c*

$$\begin{aligned} \ell_1 : x = 64 \wedge y = x \cdot z \wedge z = 2 \cdot x \\ Y := X \cdot Z \\ \ell_2 : y \cdot z = 2 \cdot x \cdot x \cdot z \end{aligned}$$

Montrer que l'annotation est correcte ou incorrecte en utilisant Frama-c

Question 6.2

$$\begin{aligned} \ell_1 : x = 10 \wedge y = z + x \wedge z = 2 \cdot x \\ y := z + x \\ \ell_2 : x = 10 \wedge y = x + 2 \cdot 10 \end{aligned}$$

Montrer que l'annotation est correcte ou incorrecte en utilisant Frama-c

Exercice 7 *annotations13.c*

Soit le contrat suivant :

```
variables X, Y, Z
requires  $x_0 \geq 0 \wedge y_0 \geq 0 \wedge z_0 \geq 0 \wedge z_0 = 25 \wedge y_0 = x_0 + 1$ 
ensures  $z_f = 100$ ;
begin
  0 :  $x^2 + y^2 = z \wedge z = 25$ ;
  (X, Y, Z) := (X+3, Y+4, Z+75);
  1 :  $x^2 + y^2 = z$ ;
end
```

Question 7.1 Traduire ce contrat avec le langage PlusCal et proposer une validation pour que ce contrat soit valide.

Question 7.2 Traduire ce contrat en ACSL et vérifier qu'il est valide ou non. S'il est non valide, proposer une correction de la pré-condition et /ou de la postcondition.

3.2 Contrat division euclidienne

Exercice 8 *division.c, division.h*

Ecrire un contrat pour la division euclidienne en tenant compte de différents cas du dénominateur.

Listing 7 – division.c

```
struct s division(int a, int b)
{
  int rr = a;
  int qq = 0;
  /*@ loop invariant b <= rr & ;
     lopp assigns rr, qq;
     while (rr >= b) { rr = rr - b; qq=qq+1;};
  resu.q= qq;
  resu.r = rr;
  return resu;
}
```

3.3 Contrat factorielle

Exercice 9 *fact.c*

Soit la fonction *fact* qui calcule la fonction factorielle.

Listing 8 – *fact.c*

```
int codefact(int n) {  
    int y = 1;  
    int x = n;  
    while (x != 1) {  
        y = y * x;  
        x = x - 1;  
    };  
    return y;  
}
```

Définir un contrat pour cette fonction en définissant la fonction mathématique factorielle.
Annoter le programme en définissant un invariant de boucle.

Exercice 10 *inv1.c*

Nous étudions ce petit algorithme qui calcule quelque chose et nous avons exécuté cet algorithme de 0 et 10 pour obtenir la suite suivante :

0 --> 0,1 --> 1,2 --> 3,3 --> 7,4 --> 15,5 --> 31,6 --> 63,7 --> 127,8 --> 255,

Listing 9 – *mathinv1.h*

```
#ifndef _A_H  
#define _A_H  
// Definition of the mathematical function mathpower2  
/*@ axiomat mathpower {  
    @ logic integer mathpower(integer n, integer m);  
    @ axiom mathpower_0: \forallall integer n; n >= 0 ==> mathpower(n,0) == 1;  
    @ axiom mathpower_in: \forallall integer n,m; n >= 0 && m >= 0  
    ==> mathpower(n,m+1) == mathpower(n,m)*n;  
    @ } */  
  
int inv1(int x);  
#endif
```

Listing 10 – *inv1.c*

```
#include <limits.h>  
#include <mathinv1.h>  
/*@ requires x >=0;  
    assigns \nothing;  
    ensures \result == mathpower(2,x)-1;  
*/  
int inv1(int x)  
{ int u=0;  
  int k=0;  
  /*@ loop invariant 0<= k && k <= x;  
    @ loop invariant u == mathpower(2,k)-1;  
    @ loop assigns u,k;  
    @loop variant x-k;  
  */
```

```

while (k < x)
{
    u=2*u+1;
    k=k+1;
};
return(u);
}

```

Si on utilise la fonction `power2`, on obtient la suite suivante :

0 --> 1, 1 --> 2, 2 --> 4, 3 --> 8, 4 --> 16, 5 --> 32, 6 --> 64, 7 --> 128, 8 --> 256, 9 -->

Question 10.1 On comprend que l'algorithme calcule la suite u_n d'entiers telle que $\forall n \in \mathbb{N} : u_n = 2^n - 1$. En particulier, $u_0 = 0$.
Donner une définition de u_{n+1} en fonction de u_n en calculant le rapport $\frac{u_{n+1}+1}{u_n+1}$

Question 10.2 Ecrire un contrat pour cet algorithme, en précisant la clause *requires* et la clause *ensures*.

Question 10.3 Proposer un invariant de boucle en vous aidant de la suite u_n et montrer qu'il est correct pour cette preuve de correction.

Question 10.4 Exprimer la terminaison de cet algorithme et justifier qu'il termine pour la précondition choisie.

4 TD8

4.1 Power2

Exercice 11 `power2.c`

Listing 11 – `qpower2.c`

```

// #include <limits.h>
/*@ axiomatic auxmath {
    @ axiom rule1: \forall int n; n > 0 ==> n*n == (n-1)*(n-1)+2*n+1;
    @ } */

/*@ requires 0 <= x;
    ensures \result == x*x;
*/
int power2(int x)
{int r,k,cv,cw,or,ok,ocv,ocw;
  r=0;k=0;cv=0;cw=0;or=0;ok=k;ocv=cv;ocw=cw;
  /*@ loop invariant cv == k*k;
    @ loop invariant k <= x;
    @ loop invariant cw == 2*k;
    @ loop invariant 4*cv == cw*cw;
    @ loop assigns k,cv,cw,or,ok,ocv,ocw; */
  while (k<x)
  {
    ok=k;ocv=cv;ocw=cw;
    k=ok+1;
    cv=ocv+ocw+1;
    cw=ocw+2;
  }
}

```

```
    r=cv;
    return(r);
}
```

Soit le fichier `qpower2.c` qui est partiellement complété et qui permet de calculer le carré d'un nombre naturel. L'exercice vise à compléter les points qui manquent puis de simplifier le résultat et de montrer l'équivalence de deux fonctions.

Question 11.1 Compléter le fichier `qpower2.c` et produire le fichier `power2.c` qui est vérifié avec *Frama-c*.

Question 11.2 Simplifier la fonction itérative en supprimant les variables commençant par la lettre `o`. Puis vérifier les fonctions obtenues avec *frama-c*.

Listing 12 – `qmainpower2.c`

```
#include <stdio.h>
#include <math.h>

int power2(int x)
{
    int r,k,cv,cw,or,ok,ocv,ocw;
    r=0;k=0;cv=0;cw=0;or=0;ok=k;ocv=cv;ocw=cw;
    while (k<x)
    {
        ok=k;ocv=cv;ocw=cw;
        k=ok+1;
        cv=ocv+ocw+1;
        cw=ocw+2;
    }
    r=cv;
    return(r);
}

int p(int x)
{
    int r;
    if (x==0)
    {
        r=0;
    }
    else
    {
        r= p(x-1)+2*x-1;
    }
    return(r);
}

int check(int n){
    int r1,r2,r;
    r1 = power2(n);
    r2 = p(n);
    if (?? == ??)
    {
        r = ??;
    }
}
```



```
    else
    { r = ??;
      };
    return r;
  }
int main()
{
    int val1, val2, val3, num;
    printf("Enter a number: ");
    scanf("%d", &num);
    val1 = power2(num);
    val2 = p(num);
    val3 = check(num);
    printf("Et le résultat pour n=%d: %d %d %d\n", num, val1, val2, val3);
    return 0;
}
```

Question 11.3 Compléter les contrats

Question 11.4 En fait, vous avez montré que les deux fonctions étaient équivalentes. Expliquez pourquoi en quelques lignes.

4.2 Mutables, pointeurs etc

Listing 13 – qpierre1.c

```
Exercise 12    //@ admit p == q;
*p = 1;
//@ assert *p + *q == 2;
```

Question 12.1 Does Frama-C prove the above snippet?

Question 12.2 Recall the weakest precondition for an assignment.

Question 12.3 Compute the subsequent verification condition for the snippet.

Question 12.4 Is the subsequent verification condition verified? If not, what to change?

Exercise 13 Here is the basic swap function (no xor trick here) :

Listing 14 – pierre2.c

```
/*@
  requires \valid(a) && \valid(b);
  assigns *a,*b;
  ensures *a == \old(*b);
  ensures *b == \old(*a);
*/
void swap(int *a, int *b){
  int tmp = *a;
  *a = *b;
  *b = tmp;
}
```

Question 13.1 Write the expected post-condition. Check it.

Question 13.2 *As warned by Frama-C, check run-time errors. Write the weakest pre-condition to pass.*

Question 13.3 *As warned by Frama-C, specify the weakest side-effects to pass.*

Question 13.4 *Also perform smoke tests.*

Exercice 14 REVERSE *Write, specify and prove the function `void reverse(int*array, size_t len)` that reverses a vector in place. Do use the previously proved swap function. Hint : Take care of the unmodified part of the vector at some iteration of the loop.*

Exercice 15 COPY

Question 15.1 *Write, specify and prove the function `void copy(int const *src, int *dst, size_t len)` that copies a range of values into another array, starting from the first cell of the array. First consider (and specify) that the two ranges are entirely separated.*

Question 15.2 *In fact, such a strong separation is not necessary. Weaken the previous one while still meeting the postcondition.*